

DP Computer Science: B 1.1.2 Computational Thinking

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define and explain the four core concepts of computational thinking:
 - Abstraction
 - Algorithmic design
 - Decomposition
 - Pattern recognition
 - Provide **real-world examples** of how these concepts are applied in various situations
 - **Analyse** a problem and identify how computational thinking concepts can be used to solve it
 - **Differentiate** between the four core concepts and explain their interrelationships
-

Key Vocabulary

Term	Definition
Computational thinking	A problem-solving approach that involves breaking down problems, identifying patterns, focusing on essential details, and creating step-by-step solutions
Decomposition	Breaking down a complex problem into smaller, more manageable parts
Pattern recognition	Identifying similarities, trends, or regularities in data or problems
Abstraction	Focusing on the essential details while ignoring irrelevant information
Algorithmic design	Creating step-by-step instructions to solve a problem

ATL Skills

Skill	Description
Thinking (Critical Thinking)	Analysing and evaluating information to identify key components and relationships
Thinking (Creative Thinking)	Generating new ideas and approaches to problem-solving
Transfer Skills	Applying computational thinking concepts to diverse contexts and real-world scenarios
Communication	Expressing computational thinking concepts clearly and effectively

2. Introduction to Computational Thinking

The Hook

"How do social media algorithms personalise your feeds?" or "How do self-driving cars navigate complex traffic situations?"

What is Computational Thinking?

Computational thinking is a problem-solving approach that involves breaking down complex problems, identifying patterns, focusing on essential details, and creating step-by-step solutions. It is a toolkit applicable to computer science and many other fields.

The Four Core Concepts

text

THE FOUR CORE CONCEPTS

DECOMPOSITION

Breaking a complex problem into smaller, more manageable parts.

PATTERN RECOGNITION

Identifying similarities, trends, or regularities in data or problems.

ABSTRACTION

Focusing on essential details while ignoring irrelevant information.

ALGORITHMIC DESIGN

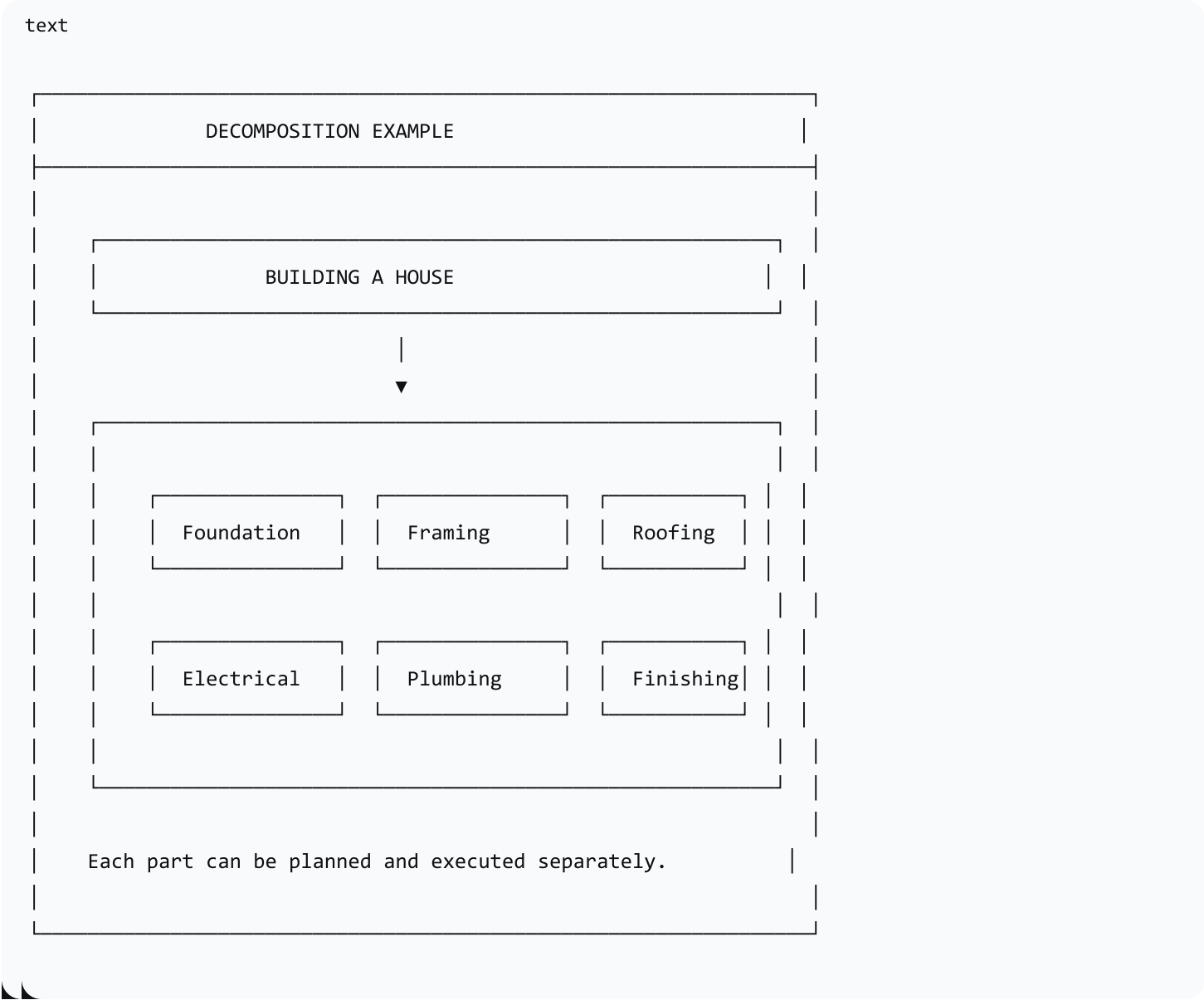
Creating step-by-step instructions to solve a problem.

3. Detailed Explanation of Each Concept

Decomposition

Decomposition is the process of breaking a complex problem into smaller, more manageable parts.

Aspect	Description
Definition	Breaking down a complex problem into smaller, more manageable subproblems
Why It Matters	Makes large problems more manageable; allows focus on one part at a time
Example	Planning a vacation: flights, accommodation, activities, packing



Real-World Example:

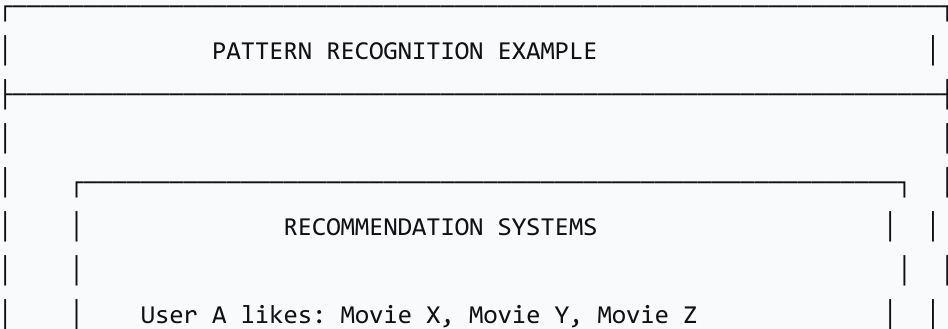
Problem	Decomposition Steps
Building a website	1. Design layout
	2. Write HTML
	3. Style with CSS
	4. Add interactivity
	5. Test and deploy
Organising an event	1. Venue
	2. Catering
	3. Entertainment
	4. Invitations
	5. Logistics

Pattern Recognition

Pattern recognition is the process of identifying similarities, trends, or regularities in data or problems.

Aspect	Description
Definition	Identifying similarities, trends, or regularities in data or problems
Why It Matters	Allows us to apply previous solutions to new problems; helps make predictions
Example	Social media algorithms identifying that users who like certain content also like similar content

text



User B likes: Movie X, Movie Y, Movie Z

Pattern: Users who like X and Y also like Z

Recommendation: If a new user likes X and Y, recommend Movie Z.

Real-World Example:

Domain	Pattern Example
Fraud Detection	Identifying unusual spending patterns
Weather Forecasting	Recognising patterns in temperature and pressure data
Healthcare	Identifying patterns in patient symptoms for diagnosis

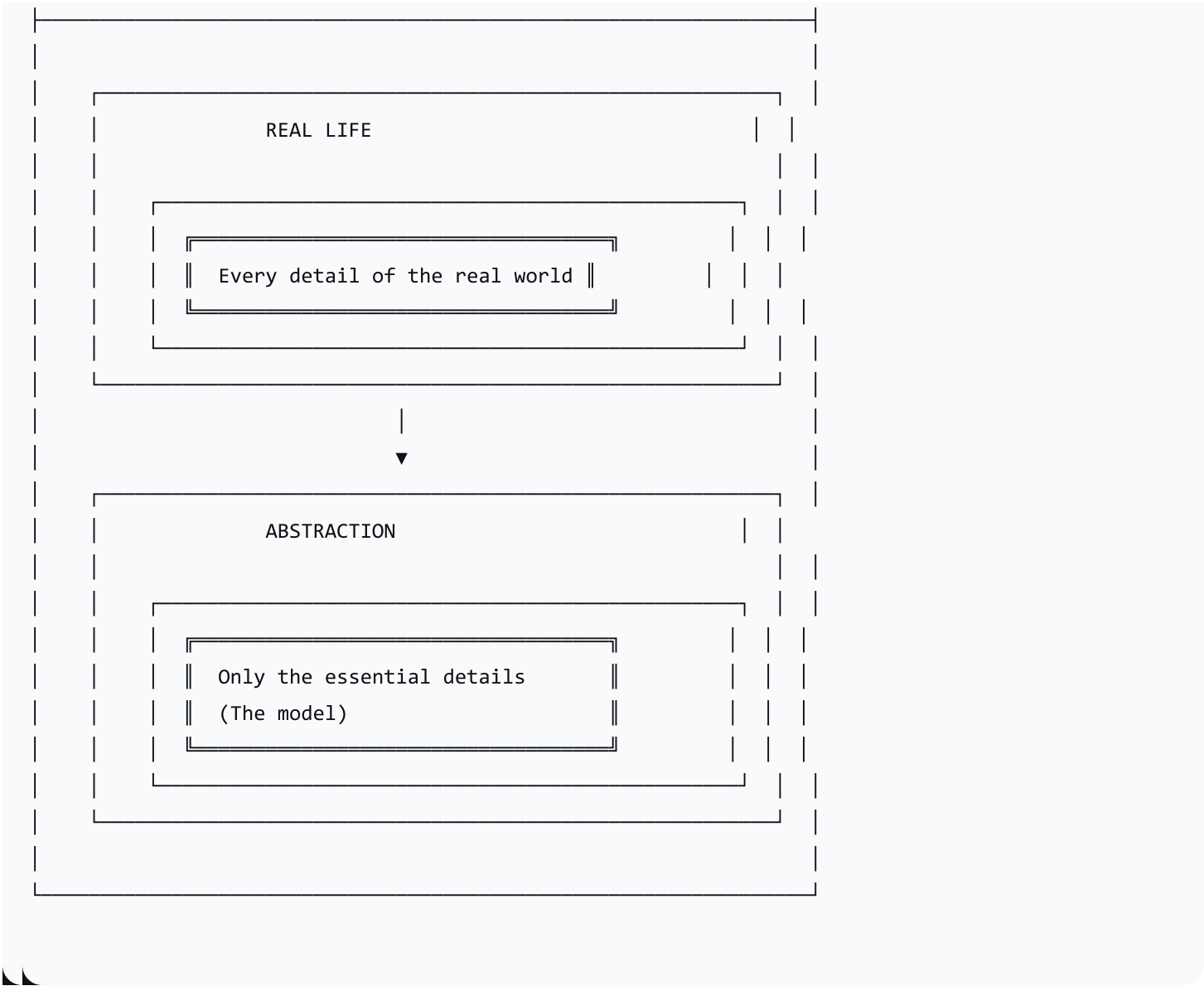
Abstraction

Abstraction is the process of focusing on the essential details while ignoring irrelevant information.

Aspect	Description
Definition	Focusing on essential details while ignoring irrelevant information
Why It Matters	Reduces complexity; helps focus on what's important
Example	A map shows only essential information (roads, landmarks) and ignores irrelevant details (individual houses)

text

ABSTRACTION EXAMPLE



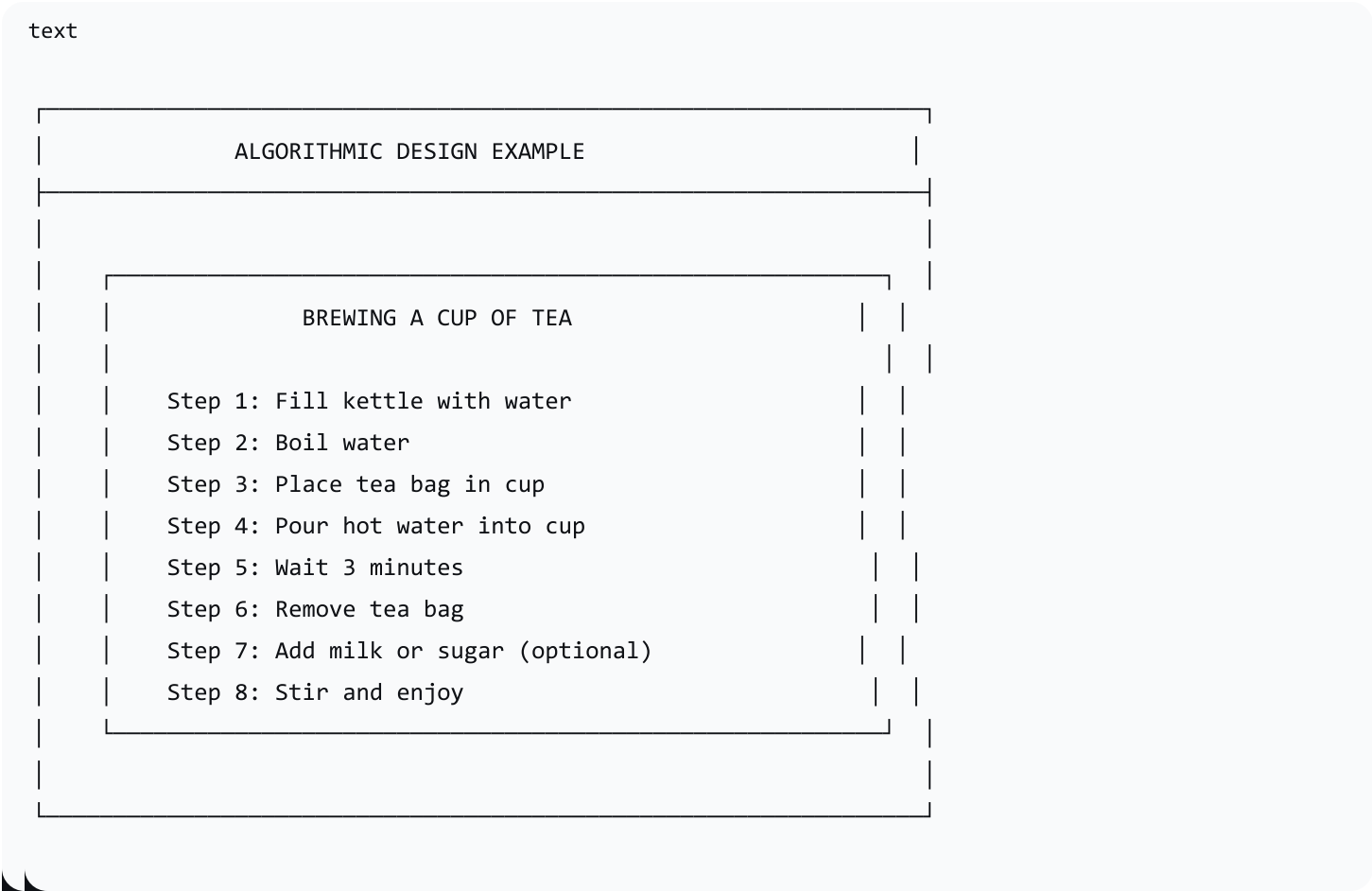
Real-World Example:

Domain	Abstraction Example
Maps	Showing only roads and landmarks; ignoring individual houses
Programming	Using functions and classes to hide complexity
Science	Creating models that represent complex systems

Algorithmic Design

Algorithmic design is the process of creating step-by-step instructions to solve a problem.

Aspect	Description
Definition	Creating step-by-step instructions to solve a problem
Why It Matters	Provides a clear, repeatable process for solving a problem
Example	A recipe for baking a cake: a sequence of steps that must be followed in order



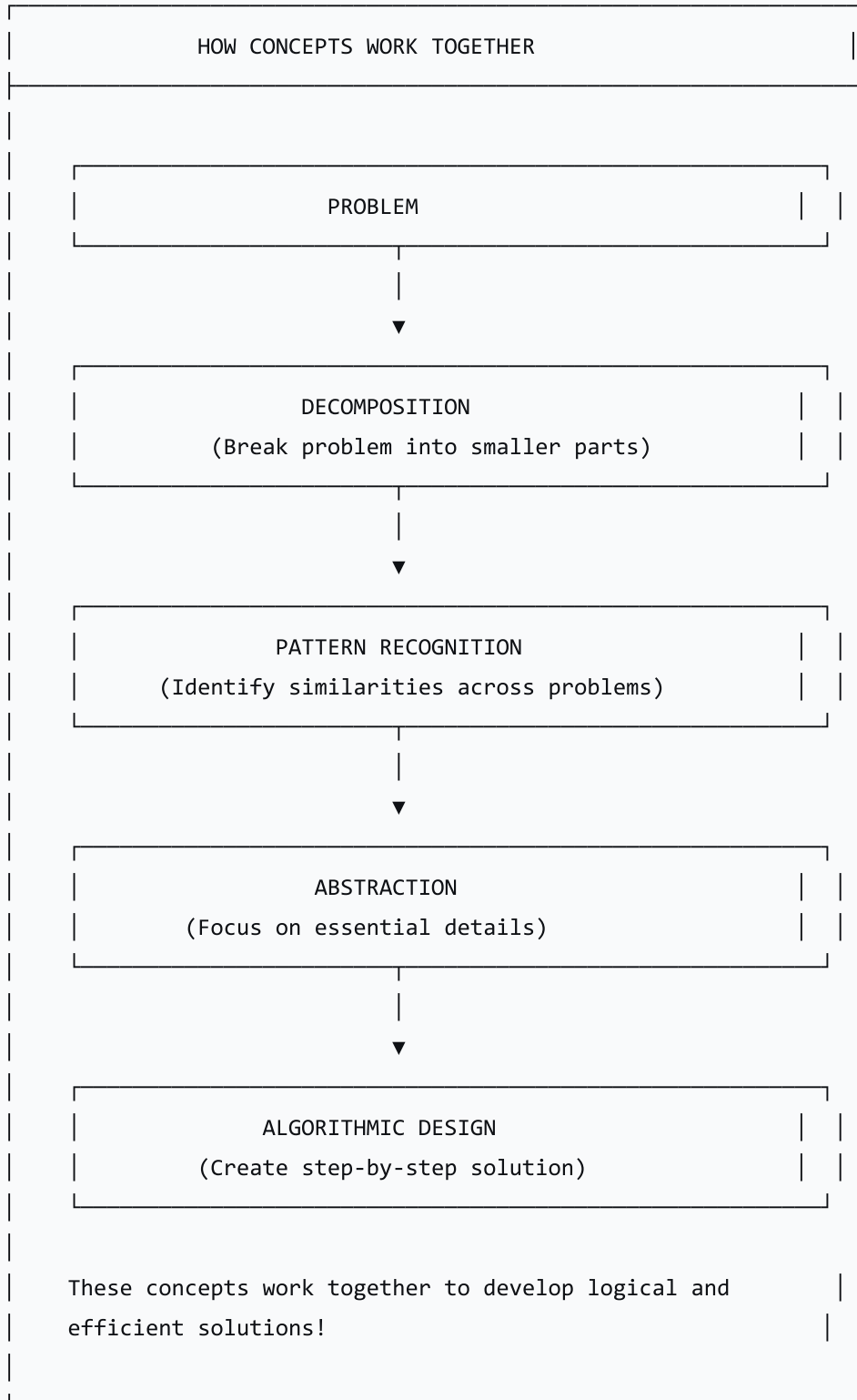
Real-World Example:

Domain	Algorithm Example
Cooking	A recipe—step-by-step instructions for preparing a dish
Navigation	Step-by-step directions for driving to a location
Programming	Code—a sequence of instructions for a computer to execute

4. The Interrelationships Between Concepts

How They Work Together

text



The Relationship Explained

Concept	Relationship
Decomposition	The starting point—break the problem down
Pattern Recognition	Find similarities—apply known solutions
Abstraction	Simplify—focus on what matters
Algorithmic Design	The final step—create the solution

5. Activities

Activity 1: Social Media Platform (Group Activity)

Scenario: A new social media platform wants to develop a feature that recommends relevant content to users based on their interests and interactions.

Challenge: Design an algorithm that considers various factors (user profile, post content, engagement metrics, trending topics) to generate personalised recommendations.

Computational Thinking Concept	Application
Decomposition	Break down the recommendation process into smaller steps: data collection, analysis, filtering, ranking
Pattern Recognition	Identify patterns in user behaviour and content preferences
Abstraction	Develop a simplified model of user interests and content relevance
Algorithmic Design	Create a step-by-step algorithm for generating recommendations

Activity 2: "Sorting Hat" Algorithm

Scenario: Design an algorithm that sorts students into groups based on their preferences for school subjects, extracurricular activities, and learning styles.

Computational Thinking Concept	Application
Decomposition	Break down the sorting process into smaller steps: data collection, analysis, sorting, evaluation
Pattern Recognition	Identify patterns in student preferences and interests
Abstraction	Develop a simplified model of student characteristics and group criteria
Algorithmic Design	Create a step-by-step process for sorting students into groups

Activity 3: Mobile Study App

Scenario: Design a mobile app to help students manage their schoolwork and study effectively.

Computational Thinking Concept	Application
Decomposition	Break down the problem into: UI design, task management, study timer, notifications, data storage
Pattern Recognition	Identify common patterns: to-do lists, reminders, user interactions
Abstraction	Focus on core features; ignore unnecessary details like specific colour schemes
Algorithmic Design	Create step-by-step instructions for each subproblem (e.g., task management algorithm)

6. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	Engaging hook; discuss role of computational thinking in technology
Key Concepts	20 min	Explain each concept with relatable examples and visuals
Activity	25 min	Group work: apply CT concepts to a real-world problem
Presentation	10 min	Groups present; peer feedback
Conclusion	5 min	Recap; real-world connections

7. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Group Activity	Assess ability to apply CT concepts
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is decomposition?
 2. What is abstraction?
 3. Give an example of pattern recognition.
 4. Why is algorithmic design important?
-

8. Differentiation

Level	Strategy
Support	Provide visual aids and relatable examples
Challenge	Explore advanced applications of CT in AI, data science

9. Resources

- Presentation (Slide Deck)
- eBook (B1.1.2 with examples)
- Quizlet (B1.1.2 vocabulary flashcards)
- Activity: Computational thinking scenarios

10. Summary: Key Takeaways

Concept	Key Point
Decomposition	Break problems down into smaller parts
Pattern Recognition	Identify similarities and trends
Abstraction	Focus on essential details
Algorithmic Design	Create step-by-step solutions

text

KEY REMINDERS	
✓	Computational thinking is a problem-solving toolkit
✓	Decomposition = break it down
✓	Pattern recognition = find similarities

- ✓ Abstraction = focus on essentials
- ✓ Algorithmic design = create step-by-step solutions
- ✓ Works together to solve problems efficiently

"Computational thinking is a powerful problem-solving approach that can be applied to any field—not just computer science."